Recap Checkpoint 4 — Cumulative: 1.1–1.5 + 2.1 (Computational Thinking) — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Q1 [3] (AO1). - Fetch (1); Decode (1); Execute (1) — must be in this order. - Fetch phase: register **used** = **PC (Program Counter)** or **MAR**; register **updated** = **MAR/MDR/CIR**, or the **PC is incremented** (accept any valid pairing as the named registers within the 3 marks — award the phase marks; named registers are credited within the description).

Examiner tip: PC holds the address of the next instruction; it is copied to the MAR, then incremented. The fetched instruction lands in the MDR and is copied to the CIR.

Q2 [4] (AO2). - (a) 205 = `1100 1101` (1). *(128+64+8+4+1 = 205)* - (b) 205 ÷ 16 = 12 remainder 13 → high nibble 12 = C, low nibble 13 = D → **CD** (1 for method, 1 for answer). - (c) Hex is **more compact / easier to read / less error-prone** — each hex digit maps to exactly 4 bits (1).

Examiner tip: cross-check (b) against (a): `1100` = C, `1101` = D → CD. The two answers must agree.

Q3 [4] (AO2). 38 − 53 in 8-bit two's complement: - (a) 38 = `0010 0110`, 53 = `0011 0101` (1 for both correct). - (b) Two's complement of 53: invert `0011 0101` → `1100 1010`, add 1 → **`1100 1011`** (1). - (c) `0010 0110` + `1100 1011` = `1111 0001` (1). `1111 0001` is negative; magnitude = invert+1 = `0000 1111` = 15, so result = **−15** (1).

Examiner tip: 38 − 53 = −15. The result `1111 0001` has a leading 1, confirming it is negative — decode it back to check.

Q4 [3] (AO1/AO2). - (a) **Agile** (accept extreme programming / iterative / rapid application development) (1). - (b) Any two (1 each): delivers **working software early/in iterations**; **adapts easily to changing requirements**; **frequent customer feedback**; less wasted effort on full up-front documentation.

Examiner tip: the cue words "requirements change frequently" and "working software early" point straight at Agile over the Waterfall model.

Q5 [4] (AO1). - (a) **Application, Transport, Network (Internet), Link (Data Link / Network Access)** — top to bottom (2 marks: 2 for all four correct/ordered, 1 for two/three correct). - (b) Routing happens at the **Network/Internet layer** (1); port numbers are added at the **Transport layer** (1).

Examiner tip: Transport = TCP/UDP and ports; Network = IP addressing and routing. Don't confuse the two.

Q6 [3] (AO3). - Legal issue (1–2): must comply with **data protection legislation (e.g. GDPR/Data Protection Act)** — keystroke data is personal data, needs a lawful basis and must be processed fairly;

collecting it secretly could be unlawful. - Ethical issue (1–2): raises concerns about **employee privacy / surveillance / trust / consent and proportionality** — monitoring may be intrusive and damage morale even if technically lawful. - Up to 3 marks total: at least one valid legal point and one valid ethical point, with reasoning.

Examiner tip: keep legal (the law) and ethical (right/wrong, fairness) distinct — examiners look for both strands.

Q7 [4] (AO2). - (a) Caching stores a **recently computed/fetched copy** of the data so repeated identical requests are served **from the cache rather than recomputing/re-querying** (1), reducing server load and **response latency** (1). - (b) Trade-off (1): the cached data can be **stale/out of date** between refreshes, so users may see slightly old temperatures; or it uses extra memory. - (c) Abstraction = **removing/hiding unnecessary detail** to focus on the essential features needed for the model (1).

Examiner tip: a cache speeds up reads but introduces a freshness trade-off — both halves are needed for full marks.

Q8 [5] (AO3). Indicative content (up to 5; reward a reasoned, applied response): - **Decomposition**: break the system into independent sub-problems — barrier control, space counting, charge calculation, app/availability service — so each can be designed, built and tested separately (1–2). - **Abstraction**: model only the essentials (e.g. represent a bay as free/occupied, ignore vehicle colour/make) so the software stays manageable (1–2). - **Concurrency**: serving many simultaneous app requests for live availability is well suited to **parallel/concurrent processing** (handling requests on separate threads), since they are largely independent (1). - Marks for clear, applied reasoning tied to the scenario rather than generic definitions.

Examiner tip: name the part suited to concurrency explicitly (the app/availability requests), not just "the whole system".

Total: 30 marks.